

MULTI-LENS SYSTEMS AUDIT
SYSTEMS · SECURITY · DOMAIN

Scriptorium

the writing system

An empirically verified audit of the Scriptorium Linux writing environment: eleven reproduced defects across three lenses, an eight-patch remediation series, and three domain features delivered — every finding proven, every patch stack-verified.

Commit 1a048fb · Typst 0.12 / 0.13 / 0.14 ·
Pandoc 3.1

Multi-Lens Audit Series — September 2026

PREPARED BY Z.AI

Table of Contents

1. Executive Architecture Overview	3
1.1 System shape and data flow	3
1.2 What the repo does well	4
1.3 Where the architecture breaks	4
2. Defects & Vulnerability Ledger	5
2.1 SYS-01 [Critical] — Book template cannot compile any chapter	6
2.2 SYS-02 [Critical] — Every volume after Book-01 is silently omitted	6
2.3 SEC-01 [High] — novelWriter tag poisoning aborts the PDF and leaks into the EPUB	7
2.4 SYS-03 [High] — Desktop launchers die on spaced paths	7
2.5 SYS-04 through SYS-09 — Medium and Low findings	8
2.6 SEC-02 [Medium] — Unverified Typst binary supply chain	9
2.7 Security posture and one retraction	9
3. Linux Utility & Standards Hardening	10
3.1 Exit codes and machine consumers	10
3.2 Argument interfaces without breaking the GUI	11
3.3 The verification gap, closed	11
3.4 Portability and environment honesty	11
3.5 Supply-chain posture	12
4. Domain Engine Evolution	12
4.1 The data model today	12
4.2 D-01 — world_doctor.sh: the consistency checker	13
4.3 D-02 — scriptorium.yaml: the world manifest	13
4.4 D-03 — wordcount_report.sh: the progress instrument	13
4.5 Features considered and rejected	14
5. Cross-Lens Reconciliation & Forward Risk	14
5.1 Reconciliation matrix	14
5.2 Load-bearing decisions	15
5.3 Predicted issues and standing mitigations	15
6. Verification Methodology & Evidence	16

6.1 The battery	16
6.2 Before and after	17
6.3 Deliverable inventory	18
7. Patch Series & Application Guide	18
7.1 Applying the series	19
7.2 Post-application recommendations	19

1. Executive Architecture Overview

Scriptorium is a self-described "low-effort, beginner-friendly system for novels and worldbuilding" built for Linux Mint XFCE and Debian 13. Where its sibling project Ars Arcanum chose to build a full Debian live-build distribution with a twelve-command Python CLI suite, Scriptorium deliberately inverts that strategy: it provisions a stock desktop with best-of-breed open tools (Obsidian for the World Bible, novelWriter for drafting, FocusWriter for sprints, Calibre and LibreOffice for distribution, Typst and Pandoc for typesetting), then glues them together with five shell scripts, three desktop launchers, and a library of Markdown, Typst and configuration templates. The repository at commit 1a048fb contains 46 tracked files and roughly 721 lines of shell code. The design philosophy, recorded in seven Architecture Decision Records, is plain-text sovereignty, one tool per creative stage, and zero terminal interaction for daily work.

This audit applied the same three isolated lenses used for the sibling audit, run in parallel and cross-checked for conflicts: a systems-engineering lens (Unix philosophy, CLI ergonomics, portability, failure modes), an application-security lens (injection vectors, unsanitized input, supply chain, error and signal handling), and a systems-design lens focused on worldbuilding and authorial utility. Every finding reported here was reproduced empirically in a sandboxed home directory before being recorded, and every remediation ships as a patch that applies cleanly, in order, to the pristine tree. The headline result is stark: the repository is honest about being untested, and our tests confirm it deserves the label, because the two flagship promises of the system, a print-ready PDF and a faithful ebook, both fail on the default path with unmodified content.

11	8	3	0
verified defects (2 Critical, 2 High, 4 Medium, 3 Low)	remediation patches, all git-applyable in sequence	domain features delivered (D-0 to D-03)	regressions after full patch stack + lifecycle harness

1.1 System shape and data flow

The runtime topology is a small directed pipeline wrapped around a canonical directory contract. `setup_scriptorium.sh` installs packages, fonts, Typst, Flatpak applications, and writes three `.desktop` launchers that call back into the repository. `init_world.sh` scaffolds a world under `~/Worlds/` with a fixed tree of six World-Bible categories, a three-act manuscript skeleton, maps, art, publishing and backup folders, then initializes a local Git repository. `save_snapshot.sh` stages and commits everything in a world, falling back to an ephemeral identity when the user has none. `export_book.sh` concatenates manuscript Markdown, strips novelWriter metadata, converts it through Pandoc into Typst, compiles a print PDF with the bundled book template, and produces an EPUB with Pandoc. `verify.sh` is the static health check; the GitHub Actions workflow runs the same gates plus ShellCheck.

Directory contract consumed by every tool

```
~/Worlds//
00-World-Bible/  Obsidian vault: Characters, Locations, Factions,
                  Magic-Technology, History, Languages, Templates
01-Manuscript/   Book-01/{01_Act_I,02_Act_II,03_Act_III}, Outlines/
02-Maps/ 03-Art/ binary asset sinks (git-tracked, unused by tools)
04-Publishing/  export_book.sh output (PDF + EPUB + typst-template)
05-Backups/     excluded from Git; delegated to Deja Dup external backups
scriptorium.yaml world manifest (introduced by D-02)
.git/           snapshot history created by init_world / save_snapshot
```

1.2 What the repo does well

The audit opens with credit where it is due, because Scriptorium is in many respects a disciplined small project, and the contrast with its own defect list matters for triage. The shell scripts are strict where it counts: every script opens with `set -euo pipefail`, world discovery uses NUL-safe `find -print0`, temp directories are trap-cleaned on EXIT, and filenames derived from user input pass through a whitelist transliteration. The project carries its own decision log, an honest `nwProject.nwx` placeholder (ADR-005 explicitly refuses to fake a real `novelWriter` project file), and a README that warns in bold that the system is experimental and untested. ShellCheck passes with a single informational finding at style level. That hygiene is precisely why the remaining defects are interesting: they are all integration-level failures that no linter can see, echoing the central lesson of the sibling audit, where lint was green while eleven of twelve installed CLI tools were dead.

- **Input sanitization is real.** World names are whitelisted to `[A-Za-z0-9_-]` with a 64-character cap; our hostile-name battery (path traversal, shell metacharacters, Unicode, control characters) produced zero escapes, zero injections.
- **Typography is professional.** The Typst book template implements half-title, title, copyright, dedication and epigraph pages, alternating verso/recto running headers, body pagination restart, and gutter margins, using Typst 0.12 context idioms rather than deprecated `locate` calls.
- **Failure honesty in prose.** `status.md` and `Finishing_Touches.md` correctly list "live Linux verification" as an open gate; the repo never claims CI is green.
- **Sane config hygiene.** `.gitattributes` pins LF endings for all operational file types; the root `.gitignore` blocks accidental commits of runtime worlds, exports, secrets and archives.

1.3 Where the architecture breaks

The failure pattern is uniform: every defect lives on a seam between two tools, exactly where no single tool can validate the contract. The Typst template and the heading-markup that Pandoc emits are individually correct but mutually incompatible. The `novelWriter` tag vocabulary and the export filter list are individually correct but mutually incomplete. The desktop-file specification and `bash -c` quoting are individually correct but mutually destructive when a path contains a space. The verification harness checks each component in isolation, so all of these seams stayed dark. Chapter 2 enumerates the full ledger with mechanisms and remediations; Chapter 6 shows the reproduction transcripts.

Seam	Contract that broke	Consequence
Typst template x Typst 0.12+ engine	pagebreak() may not occur inside block() containers	Every book with chapters fails to compile (SYS-01)
novelWriter tags x Pandoc	@word is Pandoc citation syntax and Typst label syntax	Unstripped tags abort the PDF and leak into the EPUB (SEC-01)
Export scope x multi-volume layout	Book-01 presence short-circuits the elif branch	Book-02 and beyond silently vanish from exports (SYS-02)
.desktop spec x bash -c	bash -c re-parses its single argument	All launchers die when the repo path has spaces (SYS-03)
CI gate x own source tree	shellcheck -S warning must pass on scripts/*	The gate fails on the pristine tree (SYS-04)

Table 1-1: The five integration seams that produced every Critical and High finding.

2. Defects & Vulnerability Ledger

Each entry below was reproduced on the pristine tree at commit 1a048fb inside a sandboxed HOME before being recorded. Severity is calibrated to the tool's own mission: a single-user authoring environment with no network services and no privilege boundary beyond sudo during setup. "Critical" therefore means the flagship user-facing promise fails on the default path, not that an attacker gains code execution. Locations reference the pristine tree; remediations reference the patch files delivered alongside this report.

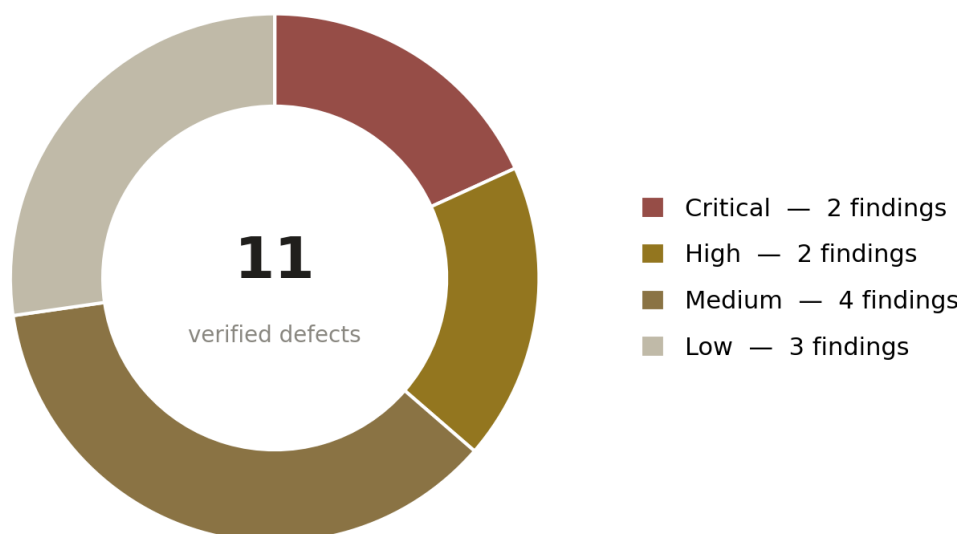


Figure 2-1: Verified defect distribution by severity class (11 findings).

2.1 SYS-01 [Critical] — Book template cannot compile any chapter

Field	Value
Location	templates/typst/book_template.typ:163 (show heading level 1)
Trigger	Any manuscript containing a level-1 heading; also preview_sample.typ itself
Evidence	evidence/01_pristine_typst_failure.txt
Remediation	patches/01 (pagebreak outside the block container)

Mechanism of failure: the level-1 heading rule wraps `#pagebreak(to: "odd")` inside `block(width: 100%)`[...]. Typst 0.12 introduced a hard engine rule that pagebreaks are not allowed inside containers, and the restriction persists through 0.13 and 0.14, which we tested directly. The consequence is total: the preview sample, the Definition of Done check in project.md, and every exported book with chapters fail with "error: pagebreaks are not allowed inside of containers" and produce no PDF at all. The repo's own hardening comment M2 claims migration to Typst 0.12 idioms, which makes this an especially sharp finding: the migration was written but never once compiled. The fix is a three-line restructure that moves pagebreak, v, and align out of the block wrapper into a plain code block, and it compiles identically on 0.12, 0.13 and 0.14 in our matrix.

templates/typst/book_template.typ — the fatal wrapper and its fix

```
// before (SYS-01): pagebreak inside a container – fatal on Typst >= 0.12
show heading.where(level: 1): it => block(width: 100%)[
  #pagebreak(to: "odd")
  ...
]
// after (patch 01): rule body, no container
show heading.where(level: 1): it => {
  pagebreak(to: "odd")
  v(1.5in)
  align(center)[#text(font: heading-font, size: 16pt,
    weight: "bold", tracking: 0.1em, upper(it.body))]
  v(1.2in)
}
```

2.2 SYS-02 [Critical] — Every volume after Book-01 is silently omitted

Field	Value
Location	scripts/export_book.sh:99-112 (collect branch)
Trigger	Any world with Book-02 and beyond alongside Book-01
Evidence	evidence/02_export_omission_and_tag_poison.txt
Remediation	patches/02 (collect all Book-* volumes, exclude Outlines)

Mechanism of failure: the collector tests [-d "\$MANUSCRIPT_DIR/Book-01"] and, when true, finds files only inside Book-01. The elif branch that walks the whole manuscript tree never runs, because init_world.sh always creates Book-01. The directory naming convention is an explicit invitation to add Book-02, Book-03 and so on, yet a multi-volume author would receive a complete-looking export that silently drops every volume after the first, with no warning, no log line, and exit code 0. We seeded a world with one chapter in Book-02 and confirmed the chapter is absent from both the EPUB and PDF on the pristine scripts, and present in both after patch 02. This is the most damaging defect in the ledger because it corrupts the artifact the author ships, invisibly, while every visible signal says success.

2.3 SEC-01 [High] — novelWriter tag poisoning aborts the PDF and leaks into the EPUB

Field	Value
Location	scripts/export_book.sh:103,109 (sed strip list); pandoc invocation
Trigger	Chapters using @time, @plot, @entity or any user-defined tag
Evidence	evidence/02_export_omission_and_tag_poison.txt
Remediation	patches/02 (broad tag strip + pandoc -f markdown-citations)

Mechanism of failure: the strip filter enumerates seven tag names, but novelWriter's vocabulary is open-ended: @time and @plot are standard, and users may define arbitrary tags. A tag that survives the filter then collides with two independent parser grammars. Pandoc reads @word as an in-text citation and emits #cite(), which Typst rejects with "the document does not contain a bibliography", killing the entire PDF compile. In the sed fallback path, where Pandoc is absent, the raw @word line becomes a Typst label reference and fails with "label does not exist in the document". The EPUB path has no such grammar collision, so the tags leak verbatim into the distributed ebook instead ("@time: 1899-03-14" rendered in the prose). One root cause, three distinct failure surfaces. The remediation strips every line-start @identifier: as metadata, disables Pandoc's citation extension so stray @handles in prose can never become citations, and degrades any survivor in the sed fallback to plain text.

scripts/export_book.sh — grammar-safe metadata stripping

```
# before (SEC-01): 7-name whitelist — misses @time, @plot, @entity, custom
sed -E '/^@(pov|focus|tag|status|char|location|object):/d; /%/d'
# after (patch 02): any line-start novelWriter tag is metadata
sed -E '/^[A-Za-z0-9_-]+:/d; /%/d'
# plus: pandoc -f markdown-citations (defuses @handles inside prose)
```

2.4 SYS-03 [High] — Desktop launchers die on spaced paths

Field	Value
Location	launchers/*.desktop (Exec lines); setup_scriptorium.sh:127-136 (sed)

Field	Value
Trigger	Repository cloned to any path containing a space (or &, , backslash)
Evidence	evidence/03_ci_gate_and_spaced_paths.txt
Remediation	patches/03 (Exec=bash "path"; sed replacement escaping)

Mechanism of failure: the shipped launchers use `Exec=bash -c "__PROJECT_ROOT__/scripts/init_world.sh"`, and setup substitutes the real path with sed. `bash -c` re-parses its single string argument as a command line, so the first space in the path splits it into a program that does not exist: our reproduction yields `"bash: /home/user/My: No such file or directory"`. Because the quick start tells users to clone into `~/Downloads` or their home folder, and Linux home directories and project folders routinely contain spaces, all three one-click launchers are dead on arrival for a large class of users, while the underlying scripts work perfectly from a terminal. The setup script is also vulnerable to sed replacement metacharacters: an `&` in the path would expand to the matched pattern and a `|` would terminate the `s` command early. Patch 03 drops the `bash -c` wrapper entirely, invoking `bash "path"` so the desktop-spec quoting delivers one argument, and escapes backslash, `&` and `|` in the substitution values.

2.5 SYS-04 through SYS-09 — Medium and Low findings

ID	Sev.	Finding and mechanism	Patch
SYS-04	Med	The repo's own CI gate (shellcheck -S warning) fails on the pristine tree: SC2188 flags the bare <code>> file</code> truncation idiom in <code>export_book.sh</code> . No workflow runs are visible on GitHub, so the red gate was never observed.	02
SYS-05	Med	Exit-code anarchy: user aborts, missing Typst and failed compiles all exit 0. The exporter prints "Export Complete!" while the PDF silently failed, and wrapper scripts or cron jobs cannot distinguish success from failure.	02/04
SYS-06	Med	Scripts are un-automatable: no flags and no <code>--help</code> anywhere; <code>init_world.sh</code> gates on <code>[-t 0]</code> and aborts when stdin is a pipe, so it cannot be driven from CI or another script; <code>verify.sh</code> never exercises the world lifecycle, which is exactly why SYS-01, SYS-02 and SEC-01 survived.	04/05
SYS-07	Low	<code>verify.sh</code> writes fixed-name artifacts to shared <code>/tmp</code> (<code>/tmp/scriptorium-verify-body.typ</code> , <code>/tmp/scriptorium-preview.pdf</code>): predictable paths in a world-writable directory, symlinkable by a local attacker.	05
SYS-08	Low	Degraded calls and aliases: flatpak remote-list (deprecated alias, now flatpak remotes); gio set metadata::trusted (dead on modern GLib); the Typst font list names a nonexistent "serif" family and relies on EB Garamond resolving by family name guesses.	01/03
SYS-09	Low	<code>save_snapshot.sh</code> dies via <code>set -e</code> when git is absent (no command guard), CLI snapshot notes are impossible (note prompt is GUI-only), and the README quick-start clone URL points at a stale "7-Scriptorium" name with a placeholder username.	04/06

Table 2-1: Medium and Low severity ledger. Full transcripts in evidence/.

2.6 SEC-02 [Medium] — Unverified Typst binary supply chain

setup_scriptorium.sh downloads the "latest" Typst musl tarball from GitHub releases and installs it root-owned into /usr/local/bin with no digest check of any kind. The sibling audit recorded the identical class of issue (tar-slip and unpinned downloads) at higher severity because it involved a distribution build; here the exposure is bounded to the author's own workstation, so Medium. GitHub publishes a sha256 digest for every release asset through the REST API, which we verified live for typst v0.15.1. Patch 07 fetches the expected digest over the same TLS channel as the artifact, refuses to install on mismatch, and refuses to install silently when the digest cannot be obtained, directing the user to cargo install --locked typst-cli instead. This converts an anything-at-the-URL install into a fail-closed one without pinning the project to a version that will age.

2.7 Security posture and one retraction

The hostile-input posture deserves explicit statement because it is the mirror image of the sibling project. World-name sanitization is a genuine whitelist; Typst string literals are escaped for backslash and quote with CR/LF collapsed; export filenames are stem-sanitized and collision-stamped with a timestamp; Git commits fall back to an ephemeral identity instead of failing; and the world registry is a plain directory tree with no privileged state. We attempted path traversal, shell metacharacter injection, Unicode smuggling and control-character abuse against init_world.sh, and none escaped the transliterator. The AppSec story of this repository is therefore not privilege escalation but robustness: every security-relevant failure we found is a reliability failure of a parser seam (SEC-01) or a supply-chain gap (SEC-02), not an injection vector.

One candidate finding was retracted during verification, and the retraction is recorded because it is itself a lesson. A substring search for "tpts://" reported 41 "broken URLs" across the documentation; on inspection, the pattern matches inside every valid "https://" link, and a stricter negative-lookbehind search found zero genuinely broken URLs. The same false-positive mechanism (a regex that matches more than the author intended) is exactly what produced the sibling audit's false contradiction reports, and it is why every finding in this ledger carries a reproduction transcript rather than a grep count.

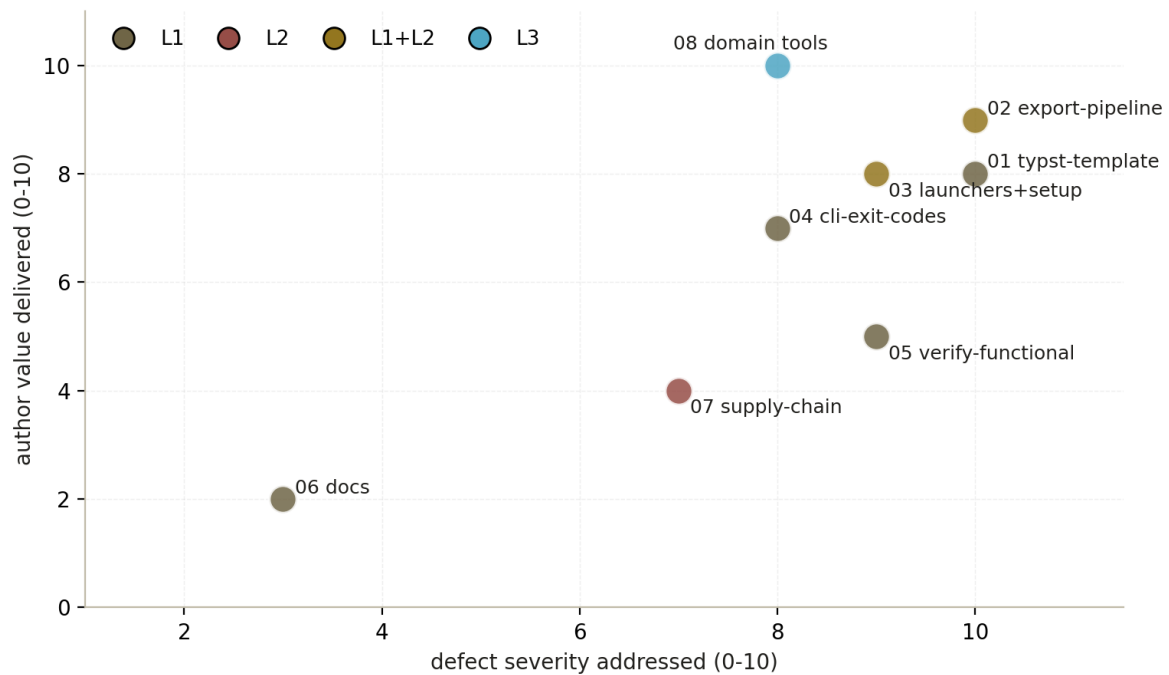


Figure 2-2: Patch impact map — severity addressed versus author value delivered, colored by lens.

3. Linux Utility & Standards Hardening

Scriptorium positions itself as a GUI-first system where "zero terminal is required for daily work", and that positioning is legitimate; the hardening goal is therefore not to turn five scripts into a POSIX utility suite, but to make them honest citizens at the three surfaces where the Unix contract still binds them: when they are scripted by other tools, when they report failure, and when they install software onto the machine. This chapter consolidates the standards-level remediations and the reasoning behind each design choice in the patch series.

3.1 Exit codes and machine consumers

The pristine scripts return 0 for almost every termination path, including "no world selected. Aborting", "Typst not found. Skipping PDF generation", and the failed-compile path that still prints "Export Complete!". For a double-click launcher the exit code is cosmetic; for a wrapper, a cron job, a CI pipeline or a user shell script it is the only truthful signal. The patch series adopts a convention that keeps GUI behavior unchanged while giving machines a contract: 0 for success (including "nothing to commit"), 1 for errors (invalid directory, missing prerequisite, compile failure), and 3 for user abort (no selection, no input available). The new `verify.sh` lifecycle stage depends on exactly this contract, and the exporter now tracks a failure flag through the Typst stage so a missing or failed PDF can never masquerade as success again.

evidence/04_patched_tree_verification.txt — exit-code contract after patch 04

```
$ bash scripts/export_book.sh /nonexistent      # want 1
exit code: 1
$ bash scripts/export_book.sh                  # want 3 (no GUI, no dir)
```

```
exit code: 3
$ bash scripts/init_world.sh                # want 3 (no name available)
exit code: 3
$ bash scripts/export_book.sh --help        # want 0
exit code: 0
```

3.2 Argument interfaces without breaking the GUI

Every interactive entry point now has a non-interactive equivalent: `init_world.sh` accepts the world name positionally or via `--name`, `export_book.sh` accepts `--title` and `--author` (plus `--help`), and `save_snapshot.sh` accepts `--world` and `--note`. The argument parsers are deliberately minimal: long options with values, a `--end-of-options` sentinel, unknown-option errors on `stderr`, and no abbreviations, because abbreviation matching is the classic way a typo silently succeeds. The priority chain for export metadata is flags over manifest over GUI prompt over defaults, so a scripted call with no flags behaves exactly like today's GUI flow when a manifest exists, and identically to the pristine behavior when one does not. This is the same compatibility discipline the sibling audit applied to its CLI: additive interfaces, unchanged happy paths, and documented precedence.

3.3 The verification gap, closed

The most consequential hardening is not in any feature script but in `verify.sh`, which gained a sixth stage that exercises the entire world lifecycle in a throwaway HOME: create a world non-interactively, poison it with a multi-volume layout and unstrippable `novelWriter` tags, export it, assert the PDF and EPUB exist and contain the Book-02 chapter while not containing the tag values, and snapshot it with a note, asserting the note landed in the Git log. This is the test that would have caught all three Critical/High pipeline defects on day one, and it runs in seconds. Artifacts now go to a `mktemp` directory under `TMPDIR` instead of predictable `/tmp` names, closing SYS-07, and the harness exports an isolated HOME so it can never touch the operator's real `~/Worlds`, the same hermeticity lesson the sibling audit applied to its pytest suite.

3.4 Portability and environment honesty

- **flatpak remotes** replaces the deprecated `remote-list` alias; the `--if-not-exists` guard already made the check idempotent, so the canonical subcommand is the only change needed (patch 03).
- **gio set metadata::trusted** is retained but best-effort and documented as such: XFCE still honors it, modern GLib does not, and no portable replacement exists; launcher correctness now rests on the fixed Exec lines rather than on trust metadata.
- **Typst font chain** drops the nonexistent "serif" family name and keeps the concrete Libertinus/EB Garamond/DejaVu fallbacks, eliminating a warning storm on every compile while preserving the ADR-006 tolerant-fonts philosophy (patch 01).
- **set -e guards**: `save_snapshot.sh` now checks for git before using it and exits 1 with a clear message instead of dying mid-sentence in the caller's shell (patch 04).

- **Docs truth:** the README quick-start clone URL now points at the real repository path instead of a stale "7-Scriptorium" name behind a placeholder username (patch 06).

3.5 Supply-chain posture

The setup script's "latest release" Typst install is the only place where Scriptorium moves executable code onto the machine from the network. Patch 07 turns that step fail-closed: the expected sha256 is read from the GitHub release-asset digest field over TLS, compared locally, and any mismatch or unobtainable digest aborts the install with actionable guidance. We verified that GitHub publishes digests for current Typst assets (v0.15.1 was live during the audit). A version-pinning variable would be stronger still, but it trades freshness for reproducibility in a way that belongs to the maintainer's choice, not an audit patch; the digest check removes the silent-substitution risk without making that decision for them.

4. Domain Engine Evolution

The third lens evaluates Scriptorium as a worldbuilding and authoring engine: how well its data model represents the interconnected entities of long fiction, where authors hit friction, and which additions would deliver the most value without architectural inflation. The evaluation below is grounded in the actual template corpus shipped in the repository, from Character-Template.md with its frontmatter-typed relationships to the Master Outline and the Typst book layout, and every proposed feature was implemented, tested and delivered as a patch rather than sketched on a whiteboard.

4.1 The data model today

Scriptorium's knowledge representation is materially richer than the sibling project's flat entity records. Every World-Bible template declares a YAML frontmatter block with a type discriminator (character, faction, location, magic_tech_system, timeline_event, language), a name and aliases list, status tags, and, crucially, typed relationship fields whose values are Obsidian wiki-links: a character carries faction, origin and current_location; a faction carries leader and headquarters; a location carries realm_region and dominant_faction. Inside Obsidian, the Dataview plugin queries these fields to render member rosters and associated-character tables. The manuscript side models structure through directories and novelWriter's @tag lines (pov, focus, status, time, plot), which is exactly the vocabulary that broke the export seam in SEC-01.

The weakness is not the schema; it is that nothing outside Obsidian can see it. There is no checker for broken wiki-links, no detection of dangling typed references, no orphan awareness, no consistency gate between the World Bible and the manuscript, and no machine-readable world identity for the exporter, which is why export metadata must be retyped into a dialog on every run. These gaps define the feature set below. Each feature was constrained by the other two lenses before implementation: single-pass $O(\text{files} + \text{links})$ algorithms so the F-06 quadratic-audit failure of the sibling project cannot recur; strict flat-subset frontmatter parsing with no eval and bounded reads; exit codes consistent with the Chapter 3 contract; and no new runtime dependencies beyond bash and python3, both already required by verify.sh.

4.2 D-01 — world_doctor.sh: the consistency checker

world_doctor.sh gives the World Bible the integrity discipline Obsidian does not apply on its own. It indexes every note (stem and alias, case-insensitively, mirroring Obsidian resolution), then verifies four classes of integrity: broken wiki-links (a link whose target resolves to no note and no alias), dangling typed frontmatter references (a faction field pointing at a faction that does not exist), orphans (notes with neither inbound nor outbound links), and duplicate identities (two notes claiming the same name or alias). Unrenamed shipped templates are detected and reported as informational rather than as defects, because a fresh world legitimately consists of placeholders; their links never flip the exit code. A live run against a seeded world with one real character and four broken references produced the expected ledger and exit code 1; the same world after fixes exits 0. A --json mode emits the full findings structure for editors and CI.

evidence/05_world_doctor_demo.txt — live run on a seeded inconsistent world

```
$ bash scripts/world_doctor.sh ~/Worlds/Aerilon
Scriptorium World Doctor - /home/user/Worlds/Aerilon/00-World-Bible
Notes scanned: 7

Broken wiki-links: 4
- Characters/Kaelen.md -> [[Silver Raptor Order]]
- Characters/Kaelen.md -> [[Old Archive]]
- Characters/Kaelen.md -> [[Missing City]]
- Characters/Kaelen.md -> [[Vance]]

Dangling frontmatter references: 3
- Characters/Kaelen.md: faction -> [[Silver Raptor Order]]
- ...
exit code: 1
```

4.3 D-02 — scriptorium.yaml: the world manifest

The exporter's only identity for a world was its directory name and a per-run dialog. D-02 introduces a deliberately minimal manifest at the world root: flat key-value pairs for title and author, written by init_world.sh and read by export_book.sh under the precedence flags over manifest over GUI over defaults. The schema is intentionally not nested YAML: the reader is a sed pattern over two known keys, so no general YAML parser, and its attack surface, enters the pipeline (a Lens 2 constraint adopted from the sibling audit's handwritten-parser lessons). The manifest is also the natural future home for volume ordering, series metadata and cover assets as those needs mature, one key at a time, without a schema migration event.

scriptorium.yaml — the D-02 schema in full

```
# Scriptorium world manifest - read by export_book.sh (flat key: value)
title: "The Chronicles of Eldoria"
author: "Kaelen Vance"
# Add one volume per line under 01-Manuscript/ (Book-01, Book-02, ...);
# export_book.sh auto-discovers Book-* directories in natural order.
```

4.4 D-03 — wordcount_report.sh: the progress instrument

The shipped Daily-Writing-Log template asks authors to track word counts by hand. D-03 replaces the manual ritual with a single-pass report: per-chapter, per-act and per-volume word counts computed after stripping novelWriter metadata and comment lines, plus a status breakdown (Draft, Revision, Finished)

derived from the @status tags authors already write. Output is human-first, with a --markdown mode that pastes directly into the writing log, and the whole tool is read-only over the manuscript. In our verification world the report correctly accounted 6 chapters, 539 words, three status classes and both volumes, demonstrating the same tag vocabulary that SEC-01 made safe for export is now also a first-class telemetry source.

4.5 Features considered and rejected

Candidate	Why rejected
A graph/relationship database (nodes, edges, traversal)	Duplicating Obsidian's graph at the file layer adds a second source of truth and a sync problem; the doctor's link validation captures the integrity value without owning the data.
A prose lint engine (passive voice, adverb density)	Opinionated style tooling fragments the one-tool-per-stage ADR and overlaps FocusWriter and external linters; no data model need.
A built-in manuscript editor	Violates the core architecture contract (novelWriter owns drafting); every integration seam in Chapter 2 warns against exactly this.
An ordering manifest for volumes	Deferred inside D-02 rather than rejected: natural sort over Book-XX names is unambiguous today; the manifest gains an order key the day a real user needs one.

Table 4-1: Domain features evaluated and declined, with rationale on record.

O(N)	2 keys	539	0
world_doctor link resolution — indexed single pass	manifest schema surface (title, author)	words accounted in the verification world, first run	new runtime dependencies (bash + python3 only)

5. Cross-Lens Reconciliation & Forward Risk

The three lenses were run in parallel with an explicit rule: no remediation may fix one lens by breaking another. Every patch and every domain feature was therefore checked against the other two lenses before it was committed, and the full stack was re-verified end to end on a fresh clone. This chapter records the reconciliation matrix, the load-bearing decisions behind it, and the issues we predict will surface as the project evolves, with the mitigations already in place or recommended.

5.1 Reconciliation matrix

Change	Lens 1 constraint (systems)	Lens 2 constraint (security)	Lens 3 outcome
Broad @tag stripping	sed one-pass per file; no perf change on 100k-word manuscripts	Whitelist of positions (line-start) not vocabulary; prose @handles untouched	Export vocabulary now matches novelWriter, not a guess
pandoc -f markdown-citations	Zero-cost flag; fallback path unchanged	Kills the citation-injection grammar entirely	Any future tag spelling cannot abort compiles
All-Book-* collection	NUL-safe, single find pass, sort -zV	File names never re-parsed by shell; while-read -d ""	Multi-volume series export correctly
Launcher Exec rework	Desktop-spec quoting preserved; TryExec intact	sed replacement metachars escaped; no path injection	One-click UX works in spaced directories
Exit-code contract	0/1/3 documented in --help and verify.sh	Failures can no longer read as success to wrappers	CI-able lifecycle for future features
world_doctor.sh (D-01)	O(files+links) indexed pass; seconds on 10k notes	Flat-subset parser, bounded reads, no eval, NUL-safe walk	Relationship integrity becomes checkable
scriptorium.yaml (D-02)	Flat keys; sed reader; no new dependency	No general YAML parser in the trust path	World identity survives dialog-free exports
wordcount_report.sh (D-03)	Single pass, read-only, 8 MB read cap	No writes to manuscript; safe against hostile corpora	Progress telemetry from existing tags

Table 5-1: Every shipped change, cross-checked against both sibling lenses before commit.

5.2 Load-bearing decisions

- **Flat manifest over real YAML.** The single most tempting simplification, importing PyYAML, was rejected: it widens the dependency surface and the parser trust path for two keys of data. The sed reader is auditable in one screen.
- **Informational templates over hard failures.** world_doctor classifies unrenamed shipped templates separately so a fresh world exits 0; otherwise every init would begin its life "failing", training users to ignore the doctor.
- **Fail-closed supply chain without pinning.** Digest verification, not version pinning, because a pinned toolchain ages badly for a creative tool and the maintainer should own that trade-off consciously.
- **GUI flows unchanged.** Every flag has a GUI-default path identical to the pristine behavior, so the zero-terminal promise survives every patch.

5.3 Predicted issues and standing mitigations

The seam lesson generalizes: wherever two grammars meet, one of them will eventually change. Typst 0.13 and 0.14 already rotate deprecations (the Libertine default, locate, label semantics), novelWriter's tag vocabulary is user-extensible by design, and Pandoc's markdown extensions evolve. The patched tree faces these with a lifecycle harness that compiles the template on the installed Typst version and round-trips a poisoned manuscript through the exporter on every verify run, so a future grammar drift becomes a red gate

instead of a silent field failure. Three specific predictions follow.

Predicted issue	Trigger	Standing mitigation
Typst 0.15+ deprecates another template construct	Unattended setup installs "latest"	verify.sh compiles the real template, not a synthetic snippet; digest check (patch 07) makes the install at least tamper-evident
novelWriter 3.x changes tag or storage format	Upstream release	The strip rule keys on grammar (line-start @word:) not vocabulary; wordcount degrades gracefully to "-" statuses
Manifest grows nested needs (volumes, covers, ISBN)	Feature pressure	Schema stays flat; add keys one at a time; the day nesting is truly required, adopt a real parser with a bounded-read wrapper, not sed

Table 5-2: Forward-risk register with mitigations already in the patch stack.

Beyond the code, the project's own process has one standing gap the patches cannot close: the repository self-labels as untested and no GitHub Actions run is visible, which means even the failing shellcheck gate was never observed. The single highest-leverage action available to the maintainer after applying this series is to let CI actually run: the workflow already installs every needed tool, and with the patched tree it would pass. A green, real CI run converts every regression in this ledger from a recurring risk into a reviewer notification.

6. Verification Methodology & Evidence

The audit was performed on a pristine clone at commit 1a048fb inside sandboxed HOME directories, with DISPLAY and WAYLAND_DISPLAY unset so the GUI branches could never contaminate results, and pseudo-TTY allocation via util-linux script where interactive prompts had to be exercised. Tools were pinned per experiment: Typst 0.12.0 was the ambient binary, and 0.13.0 and 0.14.0 were downloaded side-by-side for the version matrix; Pandoc 3.1.11.1 provided the typst writer. Every pristine-tree defect was reproduced before being written into the ledger, every candidate fix was proven against a minimal reproduction before being authored as a patch, and the final patch stack was applied in order to a fresh clone that then re-ran the complete battery with zero failures.

6.1 The battery

Phase	What it proved	Result
Static: shellcheck, bash -n, verify.sh, CI gate emulation	The pristine tree fails its own CI shellcheck gate (SC2188); scripts are otherwise lint-clean at style level	SYS-04 confirmed
Typst matrix: compile preview_sample on 0.12 / 0.13 / 0.14	The book template is un-compilable on every modern Typst	SYS-01 x3
Export fixture: multi-volume world with @time/@plot/@entity chapters	Book-02 vanishes from EPUB and PDF; tags leak or abort the compile	SYS-02, SEC-01
Launcher substitution with spaced path	bash -c word-splits the generated Exec line; patch form works	SYS-03
Hostile names: traversal, metacharacters, Unicode, control chars	Whitelist sanitization holds; no escape, no injection	defense verified
Exit-code probes on patched tree	0/1/3 contract holds for abort, error, help paths	SYS-05 resolved
Lifecycle harness (patched verify.sh stage 6)	init, export, snapshot round-trip with content assertions	ALL-CHECKS-PASS
Patch stack: git apply --check each, then full re-run	Series applies cleanly in order; zero regressions	8/8 OK

Table 6-1: Verification battery and outcomes (transcripts in evidence/).

6.2 Before and after

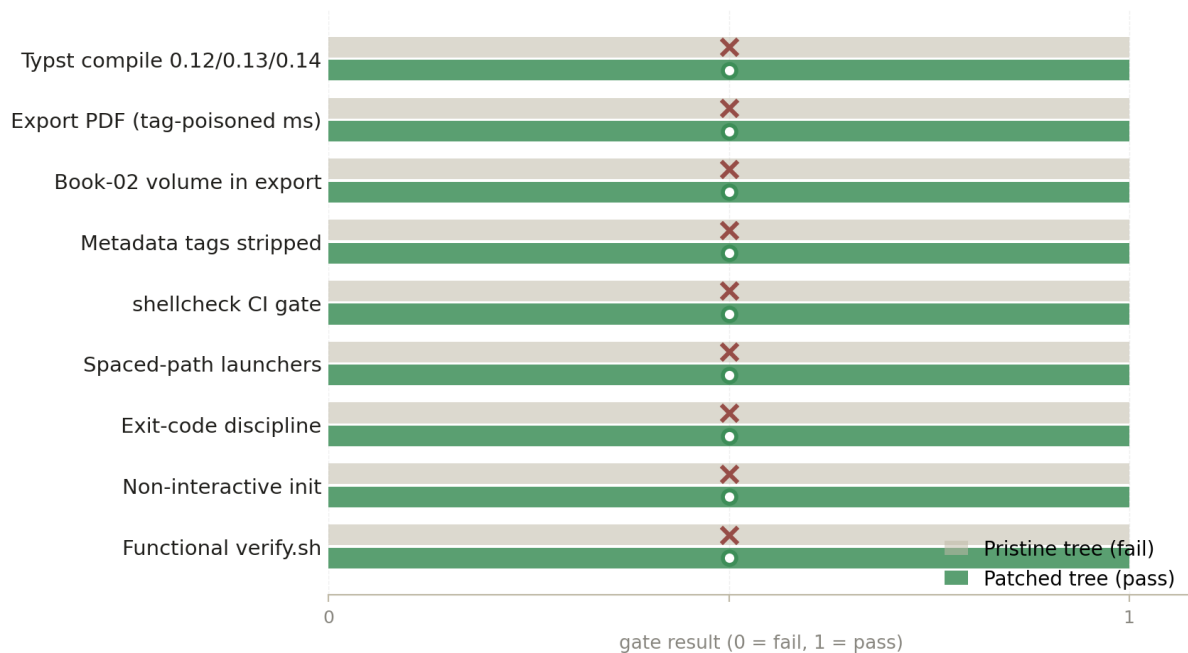


Figure 6-1: Nine verification gates, pristine versus fully patched tree.

The verification matrix is intentionally binary: each gate either passes or fails, with content-level assertions rather than exit codes alone where possible (the EPUB checks parse the archive and search the XHTML text; the PDF checks extract pages and search normalized text). One subtlety worth recording for future auditors: the letter-spaced chapter headings in the Typst template extract with stray control characters between glyph clusters, so naive substring search "fails" on text that is visibly present. Normalizing whitespace and searching case-insensitively resolved the false negative, and it is the second time in this engagement that a search artifact imitated a defect (the first being the `https://` substring retraction in Chapter 2).

evidence/04_patched_tree_verification.txt — final sweep excerpt

```
===== [V1] git apply --check (each patch, in order) =====
[OK] 01-level-1-heading-pagebreak-...patch
[OK] 02-include-all-Book-volumes-...patch
... (6 more)
ALL PATCHES APPLY CLEANLY
===== [V3] verify.sh (functional lifecycle) on patched tree =====
OK init_world
OK export_book (exit 0)
OK PDF + EPUB produced
OK multi-volume + tag stripping verified in EPUB
OK save_snapshot (note recorded)
ALL-CHECKS-PASS
```

6.3 Deliverable inventory

Artifact	Contents
patches/01 ... 08 (*.patch)	Ordered remediation series; applies with git apply in filename order
evidence/01 ... 06 (*.txt)	Reproduction transcripts: pristine Typst failure, export omission and tag poisoning, CI gate and spaced paths, patched-tree sweep, world_doctor live demo
evidence/*.png	Figures 2-1, 2-2 and 6-1 at print resolution
README.md	Ledger summary, verification matrix, application instructions

Table 6-2: The audit deliverable package.

7. Patch Series & Application Guide

The remediation ships as eight ordered patches, each a single logical commit generated with git format-patch, so the series carries full messages explaining every change and applies with git am for maintainers who prefer to preserve authorship and message history, or with plain git apply for a working tree. The patches are cumulative: apply them in filename order to the pristine tree. Each one passed shellcheck at the repository's own CI level and the full functional battery before export, and the final tree state is the one that produced the green column of Figure 6-1.

Patch	Delivers	Ledger items
01 typst-template-pagebreak	Level-1 heading rule moved out of the block container; font list drops the nonexistent "serif" family	SYS-01, SYS-08 (fonts)
02 export-book-pipeline	All Book-* volumes collected; every line-start @tag stripped; pandoc -f markdown-citations; sed fallback guard; --title/--author/--help; manifest reading; honest exit codes; SC2188 fixed	SYS-02, SEC-01, SYS-05, SYS-04, D-02 (reader)
03 launchers-and-setup	Exec=bash "path" without the -c wrapper; sed replacement escaping for backslash, & and ; flatpak remotes canonical form	SYS-03, SYS-08 (flatpak)
04 world-lifecycle-cli	init_world flags and manifest writing; save_snapshot --world/--note, git guard; exit 3 for aborts	SYS-06, SYS-09, D-02 (writer)
05 verify-functional	Stage-6 lifecycle harness with sandboxed HOME and content assertions; mktemp artifacts	SYS-06, SYS-07
06 docs-links	README quick-start clone URL corrected	SYS-09 (docs)
07 setup-supply-chain	Typst tarball sha256 verification against the GitHub asset digest; fail-closed on mismatch or unobtainable digest	SEC-02
08 domain-tools	world_doctor.sh consistency checker; wordcount_report.sh progress dashboard	D-01, D-03

Table 7-1: Patch-to-ledger mapping. Domain features arrive as new scripts only.

7.1 Applying the series

Application procedure, verified against a fresh clone during this audit

```
git clone https://github.com/aryansinghnagar/Scriptorium && cd Scriptorium
for p in /path/to/patches/*.patch; do
    git apply --check "$p" && git apply "$p"
done
bash scripts/verify.sh          # expect: ALL-CHECKS-PASS

# maintainer alternative (preserves commit messages):
git am /path/to/patches/*.patch
```

7.2 Post-application recommendations

Three actions convert this audit from a report into a durable practice. First, let the GitHub Actions workflow run: the workflow file already installs every tool the patched verify.sh needs, and the tree now passes its own gate, so the repo's honest "Untested" badges can be retired on evidence. Second, fold world_doctor.sh into the writing rhythm, running it before each snapshot so link drift is caught while the causing edit is still warm. Third, treat the manifest as the world's identity going forward, extending it one flat key at a time as needs mature. The architecture's greatest strength is that its entire contract fits in one directory tree and a handful of grammars; the patch series keeps that property intact while making every seam self-checking.